EduAirControl · Installation Manual

María de los Ángeles Olaya García; Jhon Jairo Cortez Vargas

Servicio Nacional de Aprendizaje (SENA), Programa ADSO-3145556

Instructora: Karol Daniela Correa

12 de octubre de 2026

EduAirControl · Installation Manual

SENA INDUSTRIA

EDUAIRCONTROL

Installation Manual

Preparation, local deployment, mobile and IoT prototype

Program: Technologist in Software Analysis and Development

Group (Ficha): 3145556

Team members: Jhon Jairo Cortez Vargas · María de los Ángeles Olaya García

Instructor: Karol Daniela Correa

Neiva, Huila · October 5, 2026

Version 1.1

Version Control

• | Version | Date | Description | Owner |

• | --- | --- | --- | --- |

• | 1.0 | October 5, 2026 | Initial creation of the installation manual. | Development team |

• | 1.1 | October 7, 2026 | Updated ports for the microservices architecture and added OAuth2 environment variables. | Development team |

Table of Contents

• Introduction

• Objective

• Scope and warnings

• Requirements

• Obtaining the project

• Preparing the environment file

• Web installation and execution with Docker

• 7.1 Operation commands

• 7.2 Local execution without containers

• Database and migrations

• Web system verification

• Mobile installation and execution

• 10.1 Mobile API configuration

• Physical installation of the IoT prototype

• 11.1 Verified test connections

• 11.2 Circuit tests

• BLE provisioning from mobile

• Common errors and solutions

• Uninstallation and cleanup

• Glossary

1. Introduction

This manual allows you to prepare a Windows computer to install and run EduAirControl in a local development and demonstration environment. It includes the web deployment with Docker, the mobile execution with Expo and the preliminary preparation of the ESP32 prototype.

2. Objective

To provide reproducible instructions for obtaining the code, installing tools, starting the services, verifying the API, running the mobile application and performing the basic tests of the IoT circuit.

3. Scope and Warnings

The procedure is based on the delivered files. The web Docker deployment is defined. The physical installation is described as a test prototype because it does not yet contain final ingestion firmware, battery voltage or a final electrical diagram.

4. Requirements

• | Resource | Requirement |

• | --- | --- |

• | Operating system | Windows; official minimum version pending definition |

• | RAM | 8 GB or more recommended for Docker and simultaneous services |

• | Storage | Enough space for Docker images, dependencies and PostgreSQL volumes |

• | Internet | Required to clone/obtain the code and download images and dependencies |

• | Tools | Git, Docker Desktop, Node.js/npm, Visual Studio Code |

• | Mobile | Expo Go or Android/iOS device/emulator depending on the test |

• | Hardware | ESP32, DHT11, KY-038, MQ-135, breadboard, jumpers and a safe power supply |

5. Obtaining the Project

The project contains the EduAirControl folder. If you use the repository, run:

```
git clone https://github.com/Jhon-Cortez/EduAirControl.git
cd EduAirControl
```

The mobile application is delivered as a separate project. Copy or unzip the MOBILE folder into the workspace, without mixing its package.json with that of the web project.

6. Preparing the Environment File

In Web/, create a .env file based on the values in the compose file. Real credentials must not be published. For development, you can use a local configuration equivalent to the following:

```
BACKEND_PORT=8080
POSTGRES_DB=eduaircontrol
POSTGRES_USER=eduair_user
POSTGRES_PASSWORD=CHANGE_LOCALLY
JWT_SECRET=LOCAL_KEY_OF_AT_LEAST_32_CHARACTERS
ADMIN_EMAIL=admin@eduaircontrol.com
ADMIN_PASSWORD=CHANGE_LOCALLY
ADMIN_COMPANY_CODE=EDU-0001
DB_CHANGELOG_CONTEXTS=dev
VITE_API_URL=http://localhost:8080

# OAuth2 social login
GOOGLE_CLIENT_ID=your-google-client-id
GOOGLE_CLIENT_SECRET=your-google-client-secret
FACEBOOK_CLIENT_ID=your-facebook-app-id
FACEBOOK_CLIENT_SECRET=your-facebook-app-secret
```

The PostgreSQL password and the administrator password must be defined by the person performing the installation. They must not be copied into the manual as production credentials.

7. Web Installation and Execution with Docker

Docker Desktop must be running and must use the Linux engine. From the Web folder:

```
cd EduAirControl/Web
docker compose up -d --build
```

The first startup may take a while because Docker downloads images, builds the backend and the frontend, starts PostgreSQL and runs Liquibase. The defined services are backend, frontend, postgres, mailpit and jenkins.

• | Service | Access / port |

• | --- | --- |

• | Frontend | http://localhost:5173 (Vite dev) |

• | API Gateway | http://localhost:8080 |

• | ms-security | http://localhost:8081 |

• | ms-classroom-management | http://localhost:3002 |

• | ms-environment-monitoring | http://localhost:3003 |

• | ms-sensor-management | http://localhost:3004 |

• | ms-user-management | http://localhost:3007 |

• | Swagger | http://localhost:8080/swagger-ui.html |

• | Backend health | http://localhost:8080/health |

• | PostgreSQL | localhost:5432 |

• | Mailpit web | http://localhost:8025 |

• | Jenkins | http://localhost:8081 |

7.1 Operation Commands

```
docker compose ps
docker compose logs
docker compose logs backend
docker compose down
docker compose up -d --build
```

7.2 Local Execution Without Containers

This mode is recommended for development only. You can start PostgreSQL and Mailpit with Compose and run the backend/frontend separately:

```
cd EduAirControl/Web
docker compose up -d postgres mailpit

cd backend
./mvnw spring-boot:run

cd ../Front-End
npm install
npm run dev
```

On Windows you can use mvnw.cmd spring-boot:run. The Vite frontend normally opens at http://localhost:5173; the backend remains at http://localhost:8080.

8. Database and Migrations

PostgreSQL is created through the postgres container. Liquibase runs the changelogs when the backend starts. With DB_CHANGELOG_CONTEXTS=dev, development data is loaded; with an empty value, loading of demonstration data is avoided.

• Default database: eduaircontrol.

• User and password: defined by POSTGRES_USER and POSTGRES_PASSWORD.

• Exposed port: 5432.

• Persistent volume: postgres_data.

• Do not delete the volume if you want to keep the data.

9. Web System Verification

• | Test | Expected result |

• | --- | --- |

• | Containers | docker compose ps shows active services |

• | Frontend | http://localhost:3000 shows EduAirControl |

• | Backend | http://localhost:8080/health responds without error |

• | Swagger | The documentation interface opens at /swagger-ui.html |

• | Login | You can authenticate with an administrator created in the local environment |

• | Database | The backend starts without connection errors and Liquibase finishes successfully |

• | Code tests | npm run lint, npm run build and mvn test finish successfully |

10. Mobile Installation and Execution

The mobile application is located in MOBILE/EduAirControlMobile/Front-End and is built with Expo 54.0.37, React Native 0.81.5 and React 19.1.0. Its target is Android/iOS; web export is not part of the scope.

```
cd MOBILE/EduAirControlMobile/Front-End
npm install
npm start
```

From the Expo menu you can open the application on a compatible device. For native Android, npm run android is available, which may require Android Studio and a native environment. To validate the Android bundle:

```
npm run check:android
```

10.1 Mobile API Configuration

In development, the application detects the Metro host. You can force an address reachable from the phone using EXPO_PUBLIC_API_URL. On a physical device, localhost points to the phone itself and not to the computer; you must use the computer's LAN IP:

```
EXPO_PUBLIC_API_URL=http://192.168.1.X:8080 npx expo start
```

In the Android emulator, 10.0.2.2 is used as an alias for the host computer. The application uses JWT for authentication and AsyncStorage to keep the token and user.

11. Physical Installation of the IoT Prototype

The described assembly is a demonstration on a breadboard. Before powering it on, verify voltages and continuity. Do not connect the MQ-135 to the ESP32 ADC without level adaptation if its output can exceed the maximum allowed.

• | Component | Quantity | Function |

• | --- | --- | --- |

• | ESP32 | 1 | Controller and Wi-Fi connectivity |

• | Breadboard | 1 | Solderless assembly |

• | Jumpers | Several | Connections |

• | DHT11 | 1 | Air temperature and humidity |

• | KY-038 | 1 | Relative sound level |

• | MQ-135 | 1 | Gases/air quality; not a precision CO■ meter |

• | Battery | 1 | Portable power supply; voltage pending |

• | 10 kΩ and 20 kΩ resistors | To be confirmed | Possible divider for the MQ-135; final use pending |

11.1 Verified Test Connections

• | Element | Connection / note |

• | --- | --- |

• | DHT11 DATA | GPIO 5 (D5) in the test programs |

• | KY-038 AO | GPIO 34 (D34) in the test programs |

• | Serial monitor | 115200 baud |

• | DHT11 | Use the DHT sensor library and Adafruit Unified Sensor libraries |

• | MQ-135 | Final connection, power supply and divider pending confirmation |

• | ESP32 → backend | Final firmware and ingestion protocol pending confirmation |

11.2 Circuit Tests

The test files are located in iot/esp32. For the DHT11 and KY-038, you can copy the content into the Arduino IDE, install the indicated libraries, select an ESP32 board and upload the program. Open the serial monitor at 115200 baud.

• DHT11: observe temperature and humidity without "SIN DATOS" ("NO DATA") messages.

• KY-038: make a sound near the microphone and check that the AO value changes.

• MQ-135: wait for the warm-up/stabilization defined by the sensor's procedure and verify analog variation; do not automatically interpret it as CO■ ppm.

If Wi-Fi is lost, the reconnection behavior must be verified in the final firmware.

12. BLE Provisioning from Mobile

The mobile application includes Bluetooth Low Energy provisioning using react-native-ble-plx. It searches for the ESP32_Config device using a service and characteristic UUID defined in the code. The phone must grant Bluetooth permissions; Android 12 or higher requires BLUETOOTH_SCAN and BLUETOOTH_CONNECT.

```
BLE service:     4fafc201-1fb5-459e-8fcc-c5c9c331914b
Characteristic:  beb5483e-36e1-4688-b7f5-ea07361b26a8
Expected name:   ESP32_Config
```

Provisioning sends the Wi-Fi credentials as JSON encoded in Base64. Do not record or share real credentials in screenshots or documents.

13. Common Errors and Solutions

• | Problem | Probable cause | Solution |

• | --- | --- | --- |

• | Docker does not start | Docker Desktop or the Linux engine is off | Start Docker Desktop and check docker compose ps |

• | Port in use | Another process uses 3000, 8080, 5432, 8025 or 8081 | Stop the process or change the port in .env/compose |

• | Backend does not connect to PostgreSQL | Incorrect POSTGRES_ variables | Check .env and that postgres is healthy |

• | Blank frontend | VITE_API_URL unreachable or backend stopped | Check the URL, CORS and the backend logs |

• | Expo does not connect to the API | The phone uses localhost as its own host | Use EXPO_PUBLIC_API_URL with the LAN IP |

• | DHT11 without data | Incorrect pin, power supply or library | Check DATA on GPIO 5, VCC/GND and libraries |

• | Unsafe MQ-135 reading | Analog output above the ESP32 maximum | Use adaptation/divider and verify resistors |

• | BLE does not find the ESP32 | Bluetooth off, permissions or missing firmware | Turn on Bluetooth, grant permissions and verify the name/UUID |

14. Uninstallation and Cleanup

To stop the containers without deleting data:

```
docker compose down
```

To also delete the PostgreSQL volume — a destructive action for local data — use it only when you need to reset the demonstration environment:

```
docker compose down -v
```

15. Glossary

• | Term | Definition |

• | --- | --- |

• | Docker Compose | Tool for defining and running several related services. |

• | Expo | Development platform for React Native applications. |

• | Liquibase | Version control and application of database changes. |

• | Mailpit | Local SMTP server for viewing test emails. |

• | BLE | Bluetooth Low Energy, used to provision the ESP32. |

• | GPIO | Digital or analog input/output pin of the microcontroller. |

• | ADC | Analog-to-digital converter of the ESP32. |